

Looks like it is respected from a point of view of dependencies of entities to the visualization layer. Be aware that it directly imports java graphic features. So it is somehow in between: it does not rely on concrete core classes from the original codebase but it depends on java.awt concrete methods, and it is in contrast with the clean architecture principles

the application follows a different pattern: it is an Extension Objects application. However, we can identify some classes that can be seen as entities and some other that can be seen as use cases. They do not interact with each other, thus somehow respecting basic principles of the clean architecture pattern

MapModel

Rappresenta la **mappa mentale nel suo insieme**. Gestisce: - Il nodo radice e registry di tutti i nodi (ID → NodeModel) - Listener per eventi di cambiamento della mappa - Estensioni e icone globali - File URL e stato (read-only, saved, contatore modifiche)

NodeModel

Rappresenta un **singolo nodo dell'albero**. Gestisce: - Relazioni gerarchiche (parent/children) - ID univoco - Stato di piegamento (folded) - Testo, icone, viewer - Estensioni (volutamente delegate a package dedicati) - Cloni di diversi tipi (TREE, CONTENT)

SharedNodeData

Incapsula i **dati condivisi tra cloni dello stesso nodo**: - Testo (plain text + XML formattato) - Set di icone - Cronologia (history information) - Estensioni specifiche del nodo - Contenuto (userObject)

EncryptionModel

Implementa la **crittografia dei nodi** (estensione). Gestisce: - Cifratura/decifratura del contenuto - Nascondimento dei figli durante lo stato cifrato - Stato di accesso (locked/unlocked) - Password e metodo di encryption

Relazione tra loro: MapModel contiene il nodo radice → NodeModel rappresenta ogni nodo → NodeModel condivide dati tramite SharedNodeData → EncryptionModel può cifrare un NodeModel come estensione.

MapController

Controller centrale della mappa. Gestisce: - Selezione di nodi (estende SelectionController) - Operazioni di navigazione (fold/unfold, scroll) - Listener di cambiamento mappa e nodi - Coordinamento tra modello e UI - Reader/Writer per caricamento/salvataggio

MapWriter

Serializza le mappe in XML. Gestisce: - Scrittura della struttura della mappa e dei nodi - Diverse modalità di export (CLIPBOARD, FILE, EXPORT, STYLE) - Serializzazione degli attributi e del contenuto - Implementa interfacce IElementWriter e IAttributeWriter

MapReader

Deserializza le mappe da XML. Gestisce: - Lettura e parsing del formato XML - Creazione dell'albero dei nodi durante il caricamento - Supporto di hints per controllare il comportamento - Inner class **NodeTreeCreator** per la costruzione dell'albero - Implementa IElementDOMHandler

SummaryNode

Hook persistente per nodi di riepilogo. Gestisce: - Marcatura di nodi come “sommari” (summary nodes) - Relazioni tra nodi sommari e nodi raggruppati - Supporto di FirstGroupNode per organizzazione gerarchica - Utility di verifica e manipolazione (isSummaryNode(), isHidden(), getRealNode()) - Implementa PersistentNodeHook e IExtension

Relazione tra loro: `MapController` usa `MapWriter` e `MapReader` per salvare/caricare mappe (`MapModel`), e can estendere funzionalità tramite hook come `SummaryNode`.

However, even though the name suggests use case handling, they are not, since they act also at the persistent layer. Therefore, clean architecture is not respected.